

Minimum Spanning Trees

Sean

August 27, 2026



Outline

- Spanning trees and MSTs
- Kruskal's algorithm
- DSU / Union-Find
- Kruskal implementation
- Prim's algorithm
- Problem patterns



Why another graph problem?

- Shortest paths ask: how cheaply can we travel from one place to another?



Why another graph problem?

- Shortest paths ask: how cheaply can we travel from one place to another?
- MST asks: how cheaply can we connect **everything**?



Why another graph problem?

- Shortest paths ask: how cheaply can we travel from one place to another?
- MST asks: how cheaply can we connect **everything**?
- Examples: roads, cables, pipes, network links, and grouping points by connection cost



Why another graph problem?

- Shortest paths ask: how cheaply can we travel from one place to another?
- MST asks: how cheaply can we connect **everything**?
- Examples: roads, cables, pipes, network links, and grouping points by connection cost
- The graph is weighted and undirected



1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

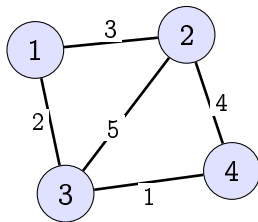
5 Prim's algorithm

6 Problem patterns



Spanning trees and MSTs

Connecting all vertices



- Vertices are objects to connect
- Edges are possible connections
- Weights are costs

Spanning tree

Definition

A **spanning tree** of a connected undirected graph is a set of edges that connects all vertices and contains no cycle.

Spanning tree

Definition

A **spanning tree** of a connected undirected graph is a set of edges that connects all vertices and contains no cycle.

- It uses exactly $n - 1$ edges

Spanning tree

Definition

A **spanning tree** of a connected undirected graph is a set of edges that connects all vertices and contains no cycle.

- It uses exactly $n - 1$ edges
- Connected: every vertex can reach every other vertex

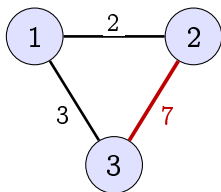
Spanning tree

Definition

A **spanning tree** of a connected undirected graph is a set of edges that connects all vertices and contains no cycle.

- It uses exactly $n - 1$ edges
- Connected: every vertex can reach every other vertex
- Acyclic: no edge is redundant

Why cycles are wasteful



- A cycle gives two ways around
- Removing one cycle edge keeps the graph connected
- In an MST, we never need a cycle

Minimum spanning tree

Definition

A **minimum spanning tree** (MST) is a spanning tree with minimum possible total edge weight.

Minimum spanning tree

Definition

A **minimum spanning tree** (MST) is a spanning tree with minimum possible total edge weight.

- It connects every vertex

Minimum spanning tree

Definition

A **minimum spanning tree** (MST) is a spanning tree with minimum possible total edge weight.

- It connects every vertex
- It uses $n - 1$ edges

Minimum spanning tree

Definition

A **minimum spanning tree** (MST) is a spanning tree with minimum possible total edge weight.

- It connects every vertex
- It uses $n - 1$ edges
- It has minimum total cost among all spanning trees

Minimum spanning tree

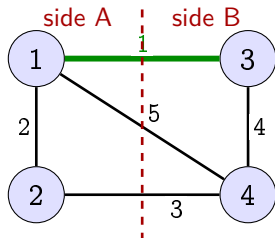
Definition

A **minimum spanning tree** (MST) is a spanning tree with minimum possible total edge weight.

- It connects every vertex
- It uses $n - 1$ edges
- It has minimum total cost among all spanning trees
- If the graph is disconnected, no spanning tree exists



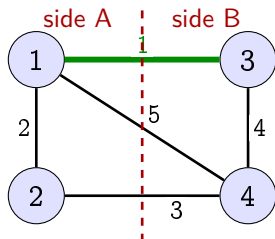
The safe edge idea



Cut property

For any cut, the cheapest edge crossing the cut is **safe**: it belongs to some MST.

The safe edge idea

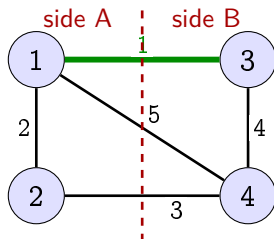


Cut property

For any cut, the cheapest edge crossing the cut is **safe**: it belongs to some MST.

- This is the correctness idea behind Kruskal

The safe edge idea



Cut property

For any cut, the cheapest edge crossing the cut is **safe**: it belongs to some MST.

- This is the correctness idea behind Kruskal
- It also explains why Prim can grow one tree outward

1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

5 Prim's algorithm

6 Problem patterns



Kruskal's algorithm

Kruskal's greedy idea

- Sort all edges by increasing weight

Kruskal's greedy idea

- Sort all edges by increasing weight
- Consider edges from cheapest to most expensive



Kruskal's greedy idea

- Sort all edges by increasing weight
- Consider edges from cheapest to most expensive
- Add an edge if it connects two different components



Kruskal's greedy idea

- Sort all edges by increasing weight
- Consider edges from cheapest to most expensive
- Add an edge if it connects two different components
- Skip an edge if it would create a cycle



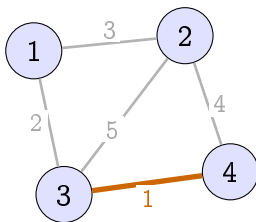
Kruskal's greedy idea

- Sort all edges by increasing weight
- Consider edges from cheapest to most expensive
- Add an edge if it connects two different components
- Skip an edge if it would create a cycle
- Stop after choosing $n - 1$ edges

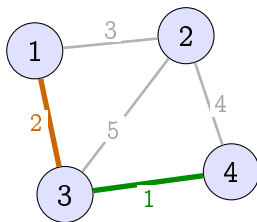


Kruskal trace

- Add edge 3 — 4 with cost 1

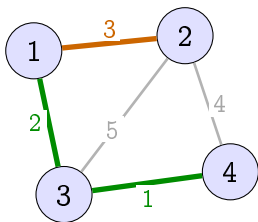


Kruskal trace



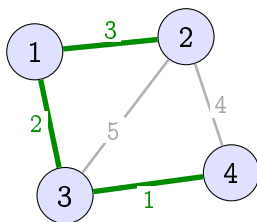
- Add edge 3 — 4 with cost 1
- Add edge 1 — 3 with cost 2

Kruskal trace



- Add edge 3 — 4 with cost 1
- Add edge 1 — 3 with cost 2
- Add edge 1 — 2 with cost 3

Kruskal trace



- Add edge 3 — 4 with cost 1
- Add edge 1 — 3 with cost 2
- Add edge 1 — 2 with cost 3
- Chosen 3 edges, so done
- Total cost: 6

The missing operation

- Kruskal repeatedly asks: are these endpoints already connected?

The missing operation

- Kruskal repeatedly asks: are these endpoints already connected?
- If yes, the edge creates a cycle

The missing operation

- Kruskal repeatedly asks: are these endpoints already connected?
- If yes, the edge creates a cycle
- If no, the edge safely merges two components

The missing operation

- Kruskal repeatedly asks: are these endpoints already connected?
- If yes, the edge creates a cycle
- If no, the edge safely merges two components
- We need a fast data structure for merging components



The missing operation

- Kruskal repeatedly asks: are these endpoints already connected?
- If yes, the edge creates a cycle
- If no, the edge safely merges two components
- We need a fast data structure for merging components
- That data structure is **DSU**, also called **Union-Find**



1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

5 Prim's algorithm

6 Problem patterns



DSU / Union-Find

DSU / Union-Find

- DSU maintains a collection of disjoint sets



DSU / Union-Find

- DSU maintains a collection of disjoint sets
- `find(x)` returns the representative of x 's set



DSU / Union-Find

- DSU maintains a collection of disjoint sets
- `find(x)` returns the representative of x 's set
- `unite(a, b)` merges the sets containing a and b



DSU / Union-Find

- DSU maintains a collection of disjoint sets
- `find(x)` returns the representative of x 's set
- `unite(a, b)` merges the sets containing a and b
- In Kruskal, each set is one connected component



DSU trace

1

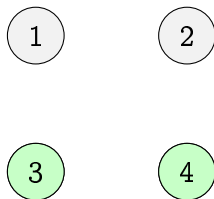
2

3

4

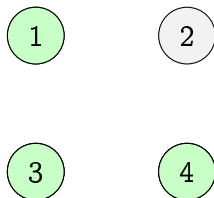
- Start:
 $\{1\}, \{2\}, \{3\}, \{4\}$

DSU trace



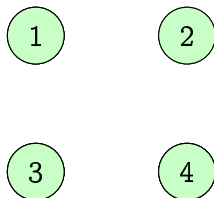
- Start:
 $\{1\}, \{2\}, \{3\}, \{4\}$
- Unite 3 and 4

DSU trace



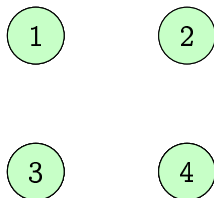
- Start:
 $\{1\}, \{2\}, \{3\}, \{4\}$
- Unite 3 and 4
- Unite 1 and 3

DSU trace



- Start:
 $\{1\}, \{2\}, \{3\}, \{4\}$
- Unite 3 and 4
- Unite 1 and 3
- Unite 1 and 2

DSU trace



- Start:
 $\{1\}, \{2\}, \{3\}, \{4\}$
- Unite 3 and 4
- Unite 1 and 3
- Unite 1 and 2
- All vertices are connected

DSU code

```
1  struct DSU {
2      vector<int> parent, sz;
3
4      DSU(int n) : parent(n + 1), sz(n + 1, 1) {
5          iota(parent.begin(), parent.end(), 0);
6      }
7
8      int find(int x) {
9          if (parent[x] == x) return x;
10         return parent[x] = find(parent[x]);
11     }
12
13     bool unite(int a, int b) {
14         a = find(a);
15         b = find(b);
16         if (a == b) return false;
17         if (sz[a] < sz[b]) swap(a, b);
18         parent[b] = a;
19         sz[a] += sz[b];
20         return true;
21     }
22 };
```



Why DSU is fast

- **Path compression:** after `find(x)`, point x directly to the representative



Why DSU is fast

- **Path compression:** after `find(x)`, point x directly to the representative
- **Union by size:** attach the smaller tree under the larger tree



Why DSU is fast

- **Path compression:** after `find(x)`, point x directly to the representative
- **Union by size:** attach the smaller tree under the larger tree
- Together, operations are nearly constant time in practice



Why DSU is fast

- **Path compression:** after `find(x)`, point x directly to the representative
- **Union by size:** attach the smaller tree under the larger tree
- Together, operations are nearly constant time in practice
- Kruskal's runtime is dominated by sorting the edges:
 $\mathcal{O}(m \log m)$



1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

5 Prim's algorithm

6 Problem patterns



Kruskal implementation

Kruskal code

```
1  struct Edge {
2      int u, v;
3      long long w;
4  };
5
6  sort(edges.begin(), edges.end(), [](Edge a, Edge b) {
7      return a.w < b.w;
8  });
9
10 DSU dsu(n);
11 long long total = 0;
12 vector<Edge> chosen;
13
14 for (Edge e : edges) {
15     if (dsu.unite(e.u, e.v)) {
16         total += e.w;
17         chosen.push_back(e);
18     }
19 }
20
21 if ((int) chosen.size() != n - 1) {
22     // No spanning tree exists.
23 }
```



Example: Road Repairation

CSES: Road Repairation

Given possible roads and their costs, find the minimum cost to connect all cities.

Example: Road Repairation

CSES: Road Repairation

Given possible roads and their costs, find the minimum cost to connect all cities.

- Weighted undirected graph

Example: Road Repairation

CSES: Road Repairation

Given possible roads and their costs, find the minimum cost to connect all cities.

- Weighted undirected graph
- Need to connect all vertices

Example: Road Reparation

CSES: Road Reparation

Given possible roads and their costs, find the minimum cost to connect all cities.

- Weighted undirected graph
- Need to connect all vertices
- Minimize total selected edge cost



Example: Road Repairation

CSES: Road Repairation

Given possible roads and their costs, find the minimum cost to connect all cities.

- Weighted undirected graph
- Need to connect all vertices
- Minimize total selected edge cost
- If Kruskal picks fewer than $n - 1$ edges, output IMPOSSIBLE



1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

5 Prim's algorithm

6 Problem patterns



Prim's algorithm

Prim's algorithm

- Kruskal grows a forest by considering edges globally



Prim's algorithm

- Kruskal grows a forest by considering edges globally
- Prim grows one connected tree outward



Prim's algorithm

- Kruskal grows a forest by considering edges globally
- Prim grows one connected tree outward
- Start from any vertex



Prim's algorithm

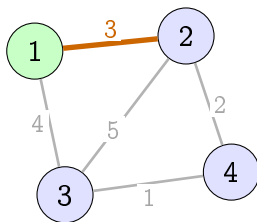
- Kruskal grows a forest by considering edges globally
- Prim grows one connected tree outward
- Start from any vertex
- Repeatedly add the cheapest edge from the tree to a new vertex

Prim's algorithm

- Kruskal grows a forest by considering edges globally
- Prim grows one connected tree outward
- Start from any vertex
- Repeatedly add the cheapest edge from the tree to a new vertex
- A priority queue tracks candidate edges

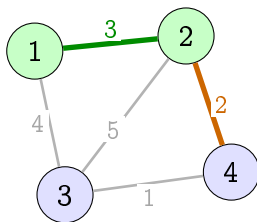


Prim trace



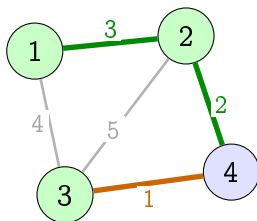
- Start from vertex 1
- Cheapest outgoing edge: 1 – 2

Prim trace



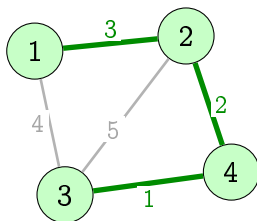
- Start from vertex 1
- Cheapest outgoing edge: 1 – 2
- From $\{1, 2\}$, choose 2 – 4

Prim trace



- Start from vertex 1
- Cheapest outgoing edge: 1 – 2
- From $\{1, 2\}$, choose 2 – 4
- From $\{1, 2, 4\}$, choose 4 – 3

Prim trace



- Start from vertex 1
- Cheapest outgoing edge: 1 – 2
- From $\{1, 2\}$, choose 2 – 4
- From $\{1, 2, 4\}$, choose 4 – 3
- Total cost: 6

Kruskal vs Prim

Algorithm	View	Main tool
Kruskal	Add cheapest safe edge globally	DSU
Prim	Grow one tree from a start vertex	Priority queue

Prim code sketch

```
1  vector<bool> used(n + 1, false);
2  using P = pair<long long, int>;
3  priority_queue<P, vector<P>, greater<P>> pq;
4  pq.push({0, 1});
5  long long total = 0;
6  int seen = 0;
7  while (!pq.empty()) {
8      auto [w, u] = pq.top();
9      pq.pop();
10     if (used[u]) continue;
11     used[u] = true;
12     total += w;
13     seen++;
14     for (auto [v, cost] : adj[u]) {
15         if (!used[v]) pq.push({cost, v});
16     }
17 }
18 if (seen != n) {
19     // No spanning tree exists.
20 }
```



1 Spanning trees and MSTs

2 Kruskal's algorithm

3 DSU / Union-Find

4 Kruskal implementation

5 Prim's algorithm

6 Problem patterns



Problem patterns

Recognizing MST problems

- The graph is undirected and weighted

Recognizing MST problems

- The graph is undirected and weighted
- You need to connect all vertices



Recognizing MST problems

- The graph is undirected and weighted
- You need to connect all vertices
- The objective is minimum total connection cost



Recognizing MST problems

- The graph is undirected and weighted
- You need to connect all vertices
- The objective is minimum total connection cost
- Cycles are unnecessary because they add extra cost without improving connectivity



Common variants

- Maximum spanning tree: sort edges descending
- Savings problems: answer is total edge cost minus MST cost
- Preconnected components: union those components before Kruskal
- Disconnected graph: no spanning tree exists
- MST may not be unique when edge weights tie



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

